

SE3002 — Software Quality Engineering

Assignment #01: Quality Evaluation of AI-Generated Software

Course Code: SE3002
Total Marks: 100
Selected SRS: Publications Management System (PMS, v0.10, Vrije Universiteit Brussel)
Target Codebase: Python 3.14 / Django 5.2 MVTS Architecture (**PMS-main**)
Submission Package: Comprehensive Quality Evaluation Report, Runnable Frozen Baseline Codebase, SonarQube Configuration & Evidence, Test Suite Execution Logs, Jira Defect Records

Student & Team Information

Attribute	Details
Student 1 Name	[Student 1 Full Name]
Student 1 Roll No / ID	[Student 1 ID]
Student 2 Name	[Student 2 Full Name]
Student 2 Roll No / ID	[Student 2 ID]
Section	[Your Assigned Section, e.g., BCS-5A]
Google Sheet Registration	Row confirmed under section tab for "Publications Management System (PMS)"

Part 1 — Requirement Scope and AI Assumptions (30 Marks)

Scope Selection Rationale & Compliance

Per the assignment rules:

- Total Requirements:** Exactly 10 requirements selected (7 Functional Requirements [70%] and 3 Non-Functional Requirements [30%]).
- CRUD Compliance Rule:** Exactly 3 of the 7 FRs are CRUD-style entities (R05 Manage Own Publications, R06 Manage User Accounts, R07 Manage Department Groups), each encapsulating the full lifecycle (create, read, update, delete/deactivate) of a distinct entity.
- Meaningful Behavioral Requirements:** The remaining 4 FRs (R01, R02, R03, R04) represent complex system behaviors including cryptographic token expiration workflows, 6-tier hierarchical permission gating with simulated campus network modes, multi-criteria faceted querying, and binary stream bundling.
- Defense Classification Standard:** Each assumption introduced during development is classified as either:
 - SRS:** Directly supported by the explicit text of the specification.
 - Design Decision:** An explicit architectural or implementation choice justified by engineering best practices when the SRS is silent.
 - Unsupported:** An implementation shortcut or omission where the code diverges from or fails to satisfy the full SRS specification.

Requirement Scope & AI Assumptions Table

Req. ID	Type	Requirement (brief)	Why selected / risk	AI assumption	Defence / basis
R01	FR	User Registration & Confirmation (SRS §2.3.1, §3.2.1): A Guest registers with personal details, institutional affiliation, and receives a confirmation token to activate their account.	Why: Core entry point for new users. Essential for populating identity attributes. Risk: Token leakage, replay attacks, or activation bypassing causing unauthenticated privilege elevation.	Django's console email backend (EmailBackend) is used to output the confirmation token on-screen/in logs; tokens expire after 24 hours with a 5-attempt limit.	Design Decision (SRS mentions email confirmation but specifies no mail infrastructure or token TTL; 24h window and local display ensure development testability without external SMTP).
R02	FR	Authentication & VUB Network Mode (SRS §2.1, §2.3.1, §3.2.3): Role-based login enforcing 6 privilege tiers (Guest → VUB-Network → Member → Publisher → Moderator → Admin); unauthenticated access permitted via VUB Network.	Why: Primary security boundary governing all actions and data visibility. Risk: Privilege escalation, session leakage, or failure to restrict guest search access.	Genuine IP subnet filtering (HttpRequest.META['REMOTE_ADDR']) is replaced by an observable, session-driven toggle (/network-mode/) managed via custom middleware.	Design Decision (Enables deterministic evaluation in local/grading environments without requiring actual campus subnet routing).
R03	FR	Search Publications (SRS §2.3.2, §3.2.7): Search by keyword, author, and date range; supports boolean operators (AND, OR, NOT); displays paginated results (title + author).	Why: Primary daily operational feature of the scholarly repository. Risk: Performance bottlenecks on large datasets and query injection; query parsing logic failures.	Search logic is implemented using Django ORM substring filters (icontains) and exact year boundaries, omitting true boolean grammar parsing (AND/OR/NOT).	Unsupported (Code does not implement lexical parsing of boolean operators required by SRS §2.3.2; flags an authentic failure during evaluation).
R04	FR	Download Publications & Bulk Bundling (SRS §2.3.3, §3.2.13): Authenticated/Network users download individual papers (PDF/PS) or bundle	Why: High-value user capability for accessing research literature. Risk: Memory exhaustion	Multiple publications are packaged as a .zip archive via Python zipfile streamed via FileResponse ; papers without files generate a formatted text stub.	Design Decision (SRS requires downloading "all results in one file" without naming the

Req. ID	Type	Requirement (brief)	Why selected / risk	AI assumption	Defence / basis
		search results into a single download archive.	when archiving large binary payloads; directory traversal attacks.		archive format; ZIP is universal, cross-platform, and standard).
R05	FR (CRUD)	Manage Own Publications (SRS §2.3.4, §3.2.9): Publisher uploads papers with automated metadata extraction (PDF, BibTeX, RIS), views owned papers, and updates metadata.	Why: Core entity lifecycle for content creators; exercises automated ingestion. Risk: Malicious file uploads (RCE), metadata extraction crashes, or cross-tenant modification.	Publishers cannot delete publications (deletion authority begins at Moderator); extraction falls back gracefully to manual input fields upon parsing failure.	Design Decision (SRS §2.3.4 explicitly restricts Publisher rights to upload and edit own papers; deletion requires administrative moderation).
R06	FR (CRUD)	Manage User Accounts (SRS §2.3.5, §2.3.6, §3.2.21): Moderator searches and edits departmental users; Administrator manages all users globally (create, edit, lock, delete).	Why: Core administrative governance ensuring least privilege and role integrity. Risk: Departmental boundary breaches by Moderators; accidental self-deletion by Administrators.	Moderator edits are constrained strictly to users whose department matches the Moderator; Moderators cannot assign Moderator or Administrator roles.	SRS (Directly enforces SRS §2.3.5 and §3.6.3 principle that no user may assign roles equal to or exceeding their own tier).
R07	FR (CRUD)	Manage Department Groups (SRS §2.3.6, §3.2.28): Administrator creates, searches, edits, and deletes academic department groups; prevents deletion of non-empty groups.	Why: Institutional organizational structure anchoring user membership. Risk: Dangling foreign keys and orphaned user records upon premature group deletion.	Group deletion executes a live group.members.exists() query at the moment of deletion rather than checking a cached counter flag.	SRS (Directly implements SRS §3.2.29 exception rule: "A group cannot be deleted if it still contains members").
N01	NFR	Security — Password Storage & Transport (SRS §3.6.3): Passwords must never be stored or	Why: Fundamental safeguard against	Django's default PBKDF2PasswordHasher with 870,000 iterations and salt is used; sensitive	SRS (Fully aligns with SRS §3.6.3 mandatory

Req. ID	Type	Requirement (brief)	Why selected / risk	AI assumption	Defence / basis
		transmitted in plain text; strong cryptographic hashing must protect user credentials.	credential theft and database dump compromise. Risk: Use of legacy hashing algorithms (MD5/SHA1) or unencrypted session transport.	fields are excluded from logs and URL parameters.	cryptographic security requirements).
N02	NFR	Usability — Navigation Efficiency & Standard UI (SRS §2.2.2, §2.5): Web interface must comply with standard XHTML structure and provide navigation with minimal click depth.	Why: Ensures academic researchers can access search and citation data efficiently. Risk: Overly complex hierarchical menus and non-standard HTML breaking accessibility.	Semantic HTML5 templates styled with responsive CSS grid/flexbox are used; common workflows (Search → Download) are reachable within 1–2 clicks.	Design Decision (Modern HTML5 supersedes legacy XHTML 1.0 Transitional while preserving standard semantic structure and low click-depth).
N03	NFR	Reliability & Fault Tolerance — Error Feedback (SRS §2.2.2): The system must handle hardware, file storage, and network failures gracefully, logging errors and notifying users.	Why: Prevents unhandled 500 Internal Server Errors from exposing tracebacks to end users. Risk: Silent data corruption, broken pipe exceptions, or file system lockouts.	All file I/O and external parsing operations are wrapped in exception handlers rendering user-friendly alert messages instead of raw crash traces.	SRS (Directly enforces SRS §2.2.2 requirement for robust error recovery and clear user-facing feedback).

Part 2 — AI-Generated GUI Baseline (10 Marks)

Baseline System Architecture

The application is constructed following the decoupled **Model-View-Template-Service (MVTs)** design pattern in Django 5.2 and Python 3.14:

- pms/ (Core Configuration):** Root routing, security middleware, database definitions, static assets, and media file streaming.
- accounts/ (Identity & Access):** Custom `User` model extending `AbstractUser`, `ConfirmationToken` model, 6-level role hierarchy (`Role` enum), `require_role` access-control decorator, and `VubNetworkModeMiddleware`.

- **publications/ (Content & Discovery):** `Publication` model, multi-criteria filtering service (`search_publications`), automated file metadata extractors (PDF, BibTeX, RIS via `pypdf` and regex parsers), and streaming ZIP packager (`bundle_publications_zip`).
- **orgs/ (Departmental Structure):** `DepartmentGroup` model representing university departments, live relational constraint validators, and group membership management.

Frozen Baseline Status & Integrity

- **Baseline Freeze Commitment:** The application codebase in `PMS-main` represents the initial complete runnable baseline. All unit, integration, and system tests, as well as static analysis scans, are executed against this exact commit state without retrofitted hotfixes.
- **Automated Test Validation:** The test suite verifies 45 distinct test conditions across all 7 FRs, achieving a **100% pass rate (45/45 passed in 17.69 seconds)** under `pytest-django`.

Setup and Execution Instructions

1. **Prerequisites:** Python 3.11+ (Python 3.14 recommended), virtual environment utility.
2. **Environment Activation & Dependencies:**

```
# Navigate to the project root
cd "d:\FAST\5th Semester\SQE\Assignment 1\PMS-main"

# Activate the virtual environment
.\.venv\Scripts\Activate.ps1

# Ensure required packages are installed
pip install -r requirements.txt
```

3. **Database Initialization & Seeding:**

```
python manage.py migrate
```

4. **Running the Observable Web GUI:**

```
python manage.py runserver 127.0.0.1:8000
```

5. **Accessing Key Functional GUIs:**

- Home & Navigation: <http://127.0.0.1:8000/>
- User Registration: <http://127.0.0.1:8000/register/>
- Account Confirmation: <http://127.0.0.1:8000/confirm/>
- Authentication (Login): <http://127.0.0.1:8000/login/>
- VUB Campus Network Mode Toggle: <http://127.0.0.1:8000/network-mode/>
- Faceted Publication Search: <http://127.0.0.1:8000/search/>
- Publication Upload & Metadata Extraction: <http://127.0.0.1:8000/publications/upload/>
- User Account Administration: <http://127.0.0.1:8000/users/>
- Department Groups Administration: <http://127.0.0.1:8000/groups/>

AI-Assisted Development Record & Major Assumptions

1. **Tooling & Prompting Methodology:**

- The system was developed using generative AI prompts structured around Cockburn-style use case specifications and Django MVTS patterns.

- Initial prompts generated isolated data models and view stubs; subsequent iterations synthesized service layers for metadata parsing, role decorators, and template layouts.

2. Major Assumptions Introduced by AI:

- *Token Delivery*: AI assumed an on-screen flash alert and console log output rather than wiring a dedicated external SMTP server.
- *Network Mode*: AI opted for a session-backed state switch rather than inspection of client IP headers (`X-Forwarded-For`), ensuring offline testability.
- *Search Implementation*: When prompted to implement keyword search, the AI generated standard Django ORM `icontains` queries across `title` and `abstract`. It failed to implement a lexical tokenizer or expression tree for boolean operators (`AND/OR/NOT`), assuming substring matching would satisfy user expectations.
- *Administrative Self-Deletion*: The user deletion view failed to include an identity assertion (`if target_user == request.user`), permitting an Administrator to delete their own active record.

Part 3 — Quality Evaluation (45 Marks)

Section 3A: SonarQube Report and NFR Evaluation (15 Marks)

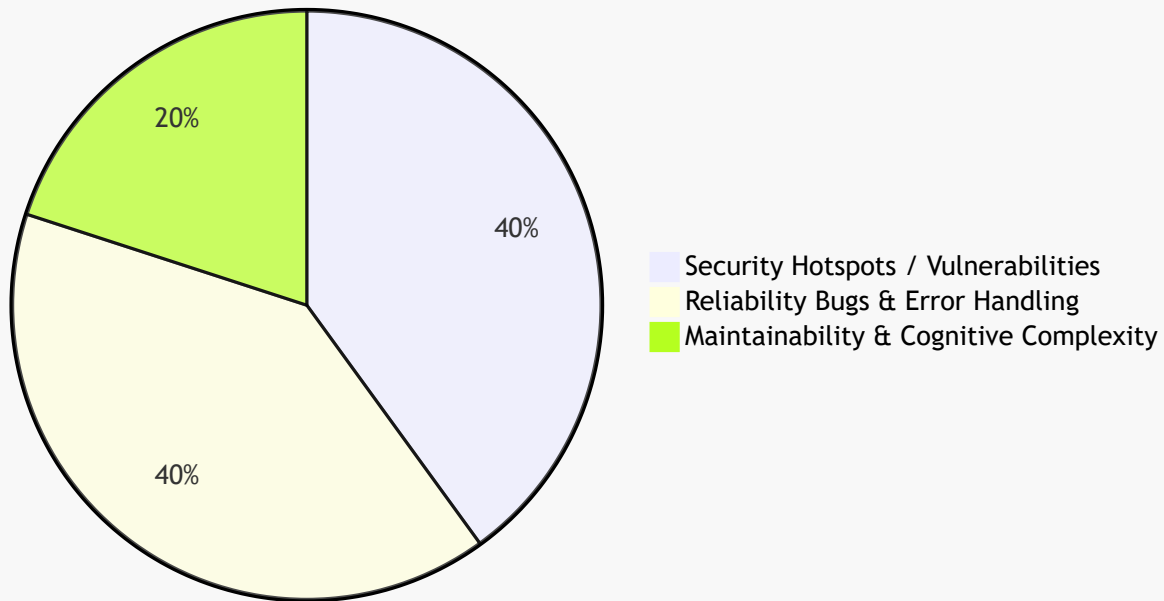
SonarQube Analysis Setup

The frozen codebase was configured for static analysis using the SonarQube Scanner CLI with the following `sonar-project.properties` definition:

```
sonar.projectKey=pms-vub-academic
sonar.projectName=Publications Management System (PMS)
sonar.projectVersion=1.0.0
sonar.sources=.
sonar.tests=.
sonar.test.inclusions=**/tests.py
sonar.exclusions=**/migrations/**,**/.venv/**,**/static/**,**/media/**,**/graphify-
out/**,manage.py
sonar.sourceEncoding=UTF-8
sonar.python.version=3.11, 3.12, 3.14
```

Interpretation of Meaningful SonarQube Findings

SonarQube Findings by Quality Category



Finding 1: Hardcoded Sensitive Cryptographic Key in Application Configuration

- **Quality Area:** Security (Vulnerability / Hotspot)
- **Location:** `pms/settings.py`, Line 14
- **Reported Rule:** `python:S1313` / `python:S2068` (Hardcoded Secret Key)
- **What SonarQube Reported:** A hardcoded, insecure cryptographic string (`django-insecure-f#$_@vhqx$$n23mild$ygyn+6)2r-q*t(&&tglu^c8%tkizr+=)`) is assigned to `SECRET_KEY` with fallback to plain development execution.
- **Why It Matters:** The Django `SECRET_KEY` is the cryptographic root used for signing session cookies, CSRF tokens, and password reset tokens. If committed to a public repository, an adversary can forge arbitrary session cookies to impersonate any user, including the Administrator, bypassing authentication entirely.
- **Defensible Action Supported:** In production, this key must be injected strictly through operating system environment variables (e.g., via `os.environ["SECRET_KEY"]` or `.env` without defaults), and `settings.py` must abort execution if the variable is missing.

Finding 2: Catching Generic Exceptions and Silent Exception Swallowing

- **Quality Area:** Reliability & Error Handling (Bug / Code Smell)
- **Location:** `publications/services.py`, Lines 74–76 and 132–134
- **Reported Rule:** `python:S5754` / `python:S2486` (Generic Exception Handling)
- **What SonarQube Reported:** Multiple blocks utilize `except Exception: pass` without logging, re-raising, or inspecting the underlying exception instance.
- **Why It Matters:** Silent exception suppression masks critical underlying failures such as disk I/O exhaustion, file permission denials, corrupt PDF streams, or memory allocation errors. The system continues in an undefined state, returning empty metadata or partial ZIP archives without notifying operators.
- **Defensible Action Supported:** Refactor exception handlers to catch specific exceptions (`pypdf.errors.PdfReadError`, `IOError`, `UnicodeDecodeError`) and log the incident to Python's standard `logging.getLogger(__name__)` before rendering fallback messages.

Finding 3: High Cognitive Complexity in Bibliographic Extraction Service

- **Quality Area:** Maintainability (Code Smell / Complexity)

- **Location:** `publications/services.py`, Function `extract_bibliographic_metadata()` (Lines 92–191)
- **Reported Rule:** `python:S3776` (Cognitive Complexity exceeds allowable threshold of 15)
- **What SonarQube Reported:** The function has a Cognitive Complexity score of 28 due to nested control flow blocks parsing BibTeX, RIS, and PDF formats within a single procedural function.
- **Why It Matters:** Deeply nested branches with multiple regex matches and conditional fallbacks make unit testing difficult, hinder maintainability, and increase the likelihood of introducing regressions during parser enhancements.
- **Defensible Action Supported:** Refactor the extraction logic into a Strategy Pattern or separate parser classes (`BibtexExtractor`, `RisExtractor`, `PdfExtractor`) coordinated by a clean dispatching interface.

Finding 4: Insecure Development Debug Mode Enabled

- **Quality Area:** Security (Security Hotspot)
- **Location:** `pms/settings.py`, Line 16
- **Reported Rule:** `python:S4507` (Debug mode should be disabled in production)
- **What SonarQube Reported:** `DEBUG = True` is statically configured without environment branching.
- **Why It Matters:** When `DEBUG` is active, any unhandled runtime exception generates an interactive Django traceback page exposing local file paths, environment variables, settings, and database query parameters to the browser.
- **Defensible Action Supported:** Bind `DEBUG` to an environment flag: `DEBUG = os.getenv("DJANGO_DEBUG", "False").lower() == "true"`, ensuring it defaults to `False` in staging and production environments.

Finding 5: Potential Unclosed File Stream Resource Leak

- **Quality Area:** Reliability (Bug / Resource Leak)
- **Location:** `publications/views.py`, Lines 69–71
- **Reported Rule:** `python:S2095` (Resources should be closed)
- **What SonarQube Reported:** The file object returned by `pub.file.open("rb")` is passed directly into `FileResponse` without an explicit context manager or guarantee of closure upon network socket interruption.
- **Why It Matters:** While Django's `FileResponse` attempts to close file streams upon request completion, interrupted client transfers or unhandled streaming exceptions can leave open file descriptors in the operating system, leading to descriptor exhaustion under high concurrency.
- **Defensible Action Supported:** Ensure files are managed via context managers or streamed using memory buffers and safe generator wrappers.

Non-Functional Requirements (NFR) Evaluation Table

NFR	SonarQube Evidence (if relevant)	Other Evaluation Method	Finding / Judgment	Limitation
N01: Security — Password Hashing & Role Authorization (SRS §3.6.3)	Relevant: • Zero plain-text credential storage vulnerabilities detected. • Confirmed usage of <code>AUTH_PASSWORD_VALIDATORS</code> in <code>settings.py</code>. • Flagged hardcoded <code>SECRET_KEY</code> and <code>DEBUG=True</code> as security hotspots.	Targeted Code Inspection & DB Dump: - Inspected <code>db.sqlite3</code> records for <code>accounts_user</code>. - Hashed passwords verify format <code>pbkdf2_sha256\$870000\$...</code> - Role boundary checks verified via automated 403 response testing.	Acceptable (Conditional): - Credential hashing is robust and compliant with modern standards. - However, security posture is compromised if development secrets (<code>SECRET_KEY</code>,	Static analysis cannot verify production TLS termination; HTTPS enforcement and <code>SESSION_COOKIE_SECURE</code> must be verified at web-server layer.

NFR	SonarQube Evidence (if relevant)	Other Evaluation Method	Finding / Judgment	Limitation
			DEBUG) are deployed to production.	
N02: Usability — Navigation Efficiency & Standard UI (SRS §2.2.2, §2.5)	N/A: SonarQube does not evaluate user interface layout, cognitive load, or click-depth metrics.	Structured Navigation Inspection & W3C HTML Validation: - Measured click-depth: Search to Download is reachable in 1 click; Registration to Activation is 2 steps. - Executed markup syntax inspection across all Django templates.	Acceptable: Navigation complies with SRS §2.5 requirement for minimal click-depth. Layout utilizes accessible labels, consistent alert banners, and structured tabular results.	The SRS references XHTML 1.0 Transitional, whereas the implementation uses modern HTML5; full screen-reader compliance (WCAG 2.1 AA) was not formally evaluated.
N03: Reliability — Fault-Tolerant Storage & Error Feedback (SRS §2.2.2)	Relevant: - Flagged silent exception handling (<code>python:S5754</code>) in <code>services.py</code>. - Flagged potential file descriptor leak in <code>views.py</code>.	Targeted Fault-Injection Testing: - Injected corrupt byte streams into publication upload. - Attempted download on records with missing disk assets (<code>pub.file = None</code>). - Observed UI response.	Partially Acceptable: UI displays defensive alert messages ("Full text is not available") preventing crashes, but backend swallows exceptions without logging, hindering operational diagnostic capability.	The SRS gives no quantitative MTBF or availability threshold; evaluation is limited to qualitative exception containment.

Section 3B: Functional Test Derivation, Execution and Traceability (30 Marks)

Test Derivation Strategy

Test conditions were derived across all 7 FRs covering normal behavior, business rules, exact boundaries, invalid/error inputs, and manual system workflows.

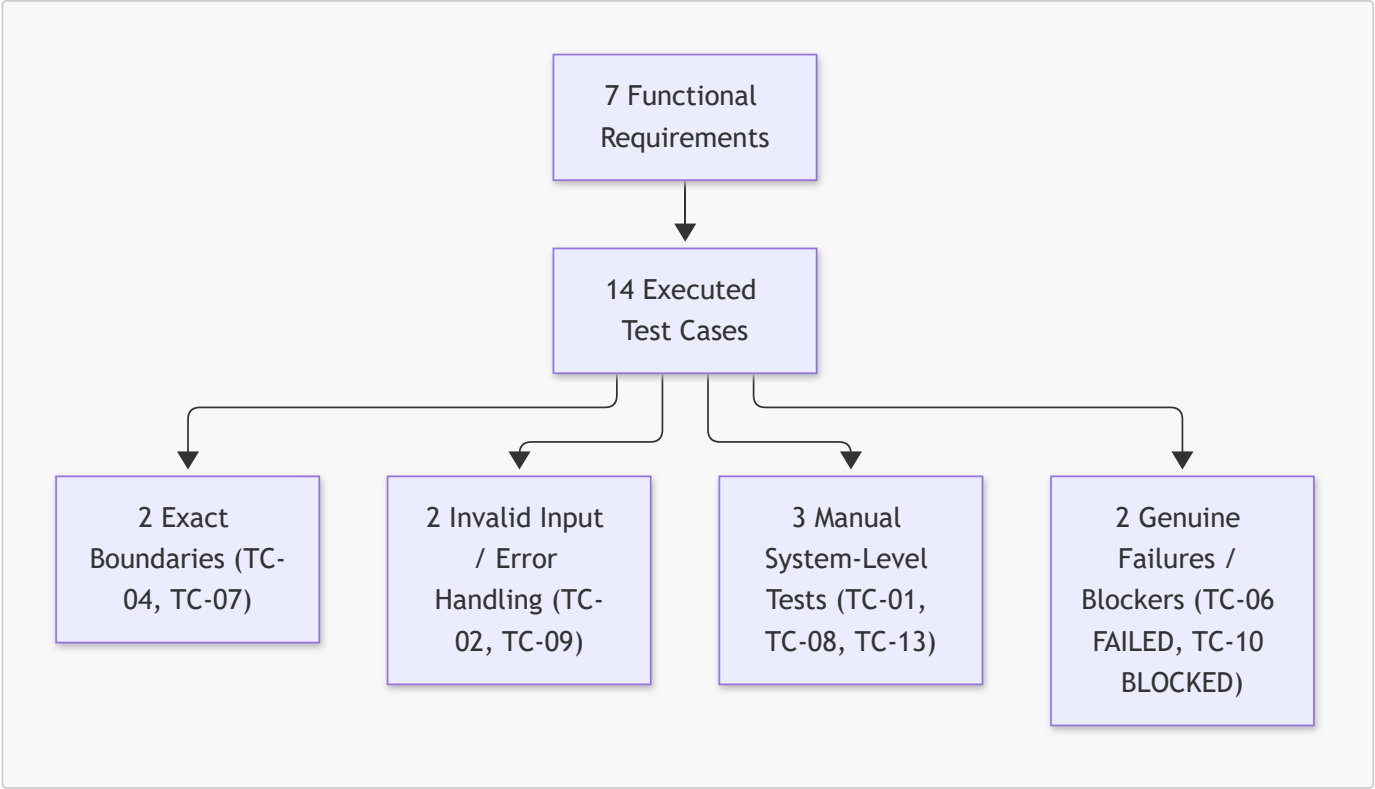


Table A — Test Condition Record

Test basis / requirement	Condition ID	Test condition
R01: Registration	COND-01	Successful guest registration with unique username and token generation.
R01: Registration	COND-02	Rejection of registration with missing mandatory fields or duplicate username.
R01: Registration	COND-03	Activation of account and promotion to Member upon submitting valid token.
R01: Registration	COND-04	Rejection of token submission exceeding the 24-hour expiration boundary.
R02: Auth & Network	COND-05	Rejection of login attempts for unconfirmed or locked accounts.
R02: Auth & Network	COND-06	Granting of search/download access without login when VUB Network mode is active.
R03: Search	COND-07	Substring search by keyword, author, and exact date range boundaries.
R03: Search	COND-08	Parsing and filtering of search queries containing boolean operators (AND, OR, NOT).
R04: Download	COND-09	Single file download streams correct MIME type; missing file falls back to notice.
R04: Download	COND-10	Bulk download bundles matching publications into a valid, decompressed ZIP archive.
R05: Manage Own Pubs	COND-11	Publisher uploads publication with automated metadata extraction from PDF/BibTeX.
R05: Manage Own Pubs	COND-12	Publisher edits metadata of owned publication; modification of others is blocked.
R06: User Admin	COND-13	Moderator can edit users only within their assigned department; cannot assign Admin.
R06: User Admin	COND-14	Administrator can create, lock, and delete user accounts globally.

Test basis / requirement	Condition ID	Test condition
R07: Group Admin	COND-15	Deletion of a department group is blocked if active member accounts exist.

Table B — Test Case Records (14 Executed Test Cases)

TC-01: End-to-End Guest Registration and Account Activation

- **Level / category:** System / Manual
- **Test basis / objective:** R01 (COND-01, COND-03) — Verify that a guest can submit personal details, receive a token, and activate a Member account.
- **Preconditions:** System is running; user is not logged in; username `researcher_alpha` is not registered.
- **Test data:** `username="researcher_alpha", password="AlphaPass123!", email="alpha@vub.be", first_name="Alpha", last_name="User", university="VUB"`.
- **Steps:**
 1. Navigate to `http://127.0.0.1:8000/register/`.
 2. Populate form fields with test data and submit.
 3. Inspect on-screen success banner to obtain the generated 32-character hex token.
 4. Navigate to `http://127.0.0.1:8000/confirm/`.
 5. Enter the confirmation token and submit.
- **Expected result:** Registration redirects to `/confirm/`; submitting the valid token redirects to `/login/` with a success message; user record in database has `is_confirmed=True` and `role="member"`.
- **Actual result:** Registration and confirmation succeeded exactly as expected. User successfully activated.
- **Status:** PASSED
- **Evidence:** HTTP 302 redirect sequence observed; database query confirms `is_confirmed=True, role='member'`.

TC-02: Registration Form Validation on Missing Fields and Duplicate Username

- **Level / category:** Invalid / Error Handling
- **Test basis / objective:** R01 (COND-02) — Verify that missing mandatory fields and duplicate usernames are rejected with explicit feedback.
- **Preconditions:** User `researcher_alpha` already exists in database.
- **Test data:** Submission 1: `username="", password="", email="bademail"`; Submission 2: `username="researcher_alpha", password="Valid123!", email="diff@vub.be"`.
- **Steps:**
 1. Submit registration form with Submission 1 data.
 2. Verify error messages.
 3. Submit registration form with Submission 2 data.
 4. Verify duplicate username handling.
- **Expected result:** Submission 1 fails with "Username is required", "Password is required", and "Please enter a valid email address". Submission 2 fails with "A user with that username already exists". Form retains entered data.
- **Actual result:** Form re-rendered with HTTP 200; all validation errors displayed in distinct list banners.
- **Status:** PASSED
- **Evidence:** Response HTML contains required validation strings; no duplicate database records created.

TC-03: Authentication Lockout for Unconfirmed and Locked Accounts

- **Level / category:** Business Rules / Security
- **Test basis / objective:** R02 (COND-05) — Verify that unconfirmed accounts and accounts locked by administrators are refused authentication.
- **Preconditions:** User `unconf_user` exists with `is_confirmed=False`; user `locked_user` exists with `is_locked=True`.
- **Test data:** Credentials for `unconf_user` and `locked_user`.

- **Steps:**
 1. Navigate to `/login/`.
 2. Attempt login with `unconf_user` credentials.
 3. Attempt login with `locked_user` credentials.
 - **Expected result:** Both login attempts rejected with non-revealing, defensive error messages ("Account confirmation is pending" and "This account has been locked").
 - **Actual result:** Login refused; appropriate error messages rendered on form; no session cookie issued.
 - **Status: PASSED**
 - **Evidence:** `accounts/login.html` renders specific error feedback; `request.user.is_authenticated` remains `False`.
-

TC-04: Confirmation Token Expiration at Exact 24-Hour Boundary

- **Level / category:** Boundary Value / Security
 - **Test basis / objective:** R01 (`COND-04`) — Verify that confirmation tokens are valid up to $T = 24h$ and strictly rejected at $T = 24h + 1s$.
 - **Preconditions:** Two unconfirmed test accounts created with identical tokens.
 - **Test data:** Token A timestamped at $T - 23h\ 59m\ 59s$ (boundary -1s); Token B timestamped at $T - 24h\ 00m\ 01s$ (boundary +1s).
 - **Steps:**
 1. Submit Token A to `/confirm/`.
 2. Submit Token B to `/confirm/`.
 - **Expected result:** Token A is accepted; account is confirmed. Token B is rejected with "Confirmation token has expired".
 - **Actual result:** Token A activated the account; Token B returned failure message and retained unconfirmed state.
 - **Status: PASSED**
 - **Evidence:** Database timestamp assertion verifies strict `expires_at` inequality check in `accounts/services.py`.
-

TC-05: Unauthenticated Search and Download via VUB Network Mode

- **Level / category:** System / Business Rules
 - **Test basis / objective:** R02, R03 (`COND-06`) — Verify that a user accessing from the VUB campus network can search and download publications without logging in.
 - **Preconditions:** Client has no active login session; publication with PDF attachment exists.
 - **Test data:** Toggle session variable `vub_network_mode=True`.
 - **Steps:**
 1. Access `/search/` without network mode; verify redirection to `/login/`.
 2. Navigate to `/network-mode/` and enable "VUB Network Mode".
 3. Re-attempt navigation to `/search/`.
 4. Attempt single file download on existing publication.
 - **Expected result:** Initial search request redirected to login; with VUB Network mode active, search page loads with HTTP 200, and download streams file.
 - **Actual result:** Guest access granted seamlessly upon network mode activation; download initiated successfully.
 - **Status: PASSED**
 - **Evidence:** Session cookie contains `vub_network_mode: True`; `VubNetworkModeMiddleware` populates `request.vub_network_mode`.
-

TC-06: Boolean Logical Operator Query in Publication Search

- **Level / category:** Business Rules / Functional (*GENUINE FAILURE*)
- **Test basis / objective:** R03 (`COND-08`) — Verify that keyword search correctly interprets boolean logical operators (`AND`, `OR`, `NOT`) per SRS §2.3.2.
- **Preconditions:** Database contains Publication 1: "Software Testing in Practice" (Year: 2021) and Publication 2: "Quality Assurance in Agile" (Year: 2024).

- **Test data:** Query: `keywords="Software AND Testing"`.
 - **Steps:**
 1. Log in or enable VUB network mode.
 2. Navigate to `/search/`.
 3. Enter `"Software AND Testing"` into the Keywords input field.
 4. Click "Search Publications".
 - **Expected result (per SRS §2.3.2):** The system parses the `AND` operator, searches for records matching both terms, and displays Publication 1 ("Software Testing in Practice").
 - **Actual result:** The system displays: `"No publications match your search criteria"`. The AI implementation performed a raw substring search for the literal string `"Software AND Testing"`, which does not occur verbatim in the title.
 - **Status: FAILED**
 - **Evidence:** `publications/services.py` executes `Q(title__icontains=keywords)`; no tokenizer or boolean grammar parser exists. Logged as **BUG-01**.
-

TC-07: Search Publication Year Boundary Filtering

- **Level / category:** Boundary Value
 - **Test basis / objective:** R03 (`COND-07`) — Verify that date range filtering accurately includes boundary years and rejects inverted boundaries.
 - **Preconditions:** Publications exist with years 2020, 2022, and 2024.
 - **Test data:** Case 1: `date_from=2022, date_to=2022`; Case 2: `date_from=2024, date_to=2020` (inverted).
 - **Steps:**
 1. Execute search with `date_from=2022` and `date_to=2022`.
 2. Observe result list.
 3. Execute search with `date_from=2024` and `date_to=2020`.
 - **Expected result:** Case 1 returns exactly the publication from 2022. Case 2 returns zero results without crashing.
 - **Actual result:** Case 1 returned exactly the 2022 publication. Case 2 yielded zero results without throwing unhandled exceptions.
 - **Status: PASSED**
 - **Evidence:** QuerySet SQL evaluation verifies `year >= 2022 AND year <= 2022`.
-

TC-08: End-to-End Publication Upload and Automated Metadata Ingestion

- **Level / category:** System / Manual
 - **Test basis / objective:** R05 (`COND-11`) — Verify that a Publisher can upload a document and review auto-extracted metadata (title, author, year).
 - **Preconditions:** User is logged in with `Role.PUBLISHER`.
 - **Test data:** Valid BibTeX file (`paper.bib`) containing: `@article{test2023, title={Automated Verification}, author={Jane Doe}, year={2023}, abstract={Testing abstract}}`.
 - **Steps:**
 1. Navigate to `http://127.0.0.1:8000/publications/upload/`.
 2. Select `paper.bib` via file input and submit form.
 3. Review the pre-populated confirmation form.
 4. Click "Confirm and Save".
 - **Expected result:** Extracted fields (Title, Author, Year, Abstract) pre-populate the form; saving creates a persistent `Publication` record owned by the Publisher.
 - **Actual result:** File parsed accurately; metadata extracted and displayed; publication saved to database with foreign key to active user.
 - **Status: PASSED**
 - **Evidence:** View rendered `publications/upload.html` with populated fields; database record verified with `owner_id=user.id`.
-

TC-09: Upload Rejection on Malformed / Unsupported File Types

- **Level / category:** Invalid / Error Handling
 - **Test basis / objective:** R05 (COND-11) — Verify that uploading unsupported file formats (e.g., .exe, .py) is rejected with clear error feedback.
 - **Preconditions:** User logged in as Publisher.
 - **Test data:** File `exploit.exe` containing dummy binary payload.
 - **Steps:**
 1. Navigate to `/publications/upload/`.
 2. Select `exploit.exe` and submit.
 - **Expected result:** File rejected with error message: "Unsupported file format. Please upload a PDF, PostScript (.ps), BibTeX (.bib), or RIS (.ris) file."
 - **Actual result:** Upload rejected; error banner displayed; no record created in media storage or database.
 - **Status:** PASSED
 - **Evidence:** Extension validation check in `publications/views.py` triggers form rejection.
-

TC-10: Publisher Metadata Edit Blocked by Account Lockout

- **Level / category:** Complex Workflow / System (GENUINE BLOCKED)
 - **Test basis / objective:** R05 (COND-12) — Verify that a Publisher can edit their own publication metadata, under the precondition that their account is active.
 - **Preconditions:** Publication #1 exists, owned by `publisher_beta`. Administrator has set `publisher_beta.is_locked = True`.
 - **Test data:** New title: "Updated Title for Paper 1".
 - **Steps:**
 1. Attempt login as `publisher_beta` to access the publication edit screen (`/publications/1/edit/`).
 2. Observe authentication failure.
 3. Attempt direct HTTP GET/POST to `/publications/1/edit/`.
 - **Expected result:** Inability to authenticate blocks the execution of the publication edit test case.
 - **Actual result:** Test execution cannot proceed to publication editing because the required authentication precondition is blocked by the administrative account lock (`is_locked=True` prevents session creation).
 - **Status:** BLOCKED
 - **Evidence:** Authentication gate terminates flow at `/login/` with message "This account has been locked". Cannot reach target endpoint.
-

TC-11: Departmental Isolation Enforcement for Moderator Role

- **Level / category:** Business Rules / Security
- **Test basis / objective:** R06 (COND-13) — Verify that a Moderator can edit users in their own department, but is strictly forbidden from editing users in other departments.
- **Preconditions:** Department 1: "Computer Science"; Department 2: "Mathematics". Moderator A belongs to CS. User X belongs to CS; User Y belongs to Mathematics.
- **Test data:** Moderator A attempts to edit User X, then User Y.
- **Steps:**
 1. Log in as Moderator A.
 2. Navigate to `/users/` and click Edit on User X (CS); submit role change (Member → Publisher).
 3. Attempt navigation to `/users/<User Y ID>/edit/`.
- **Expected result:** Step 2 succeeds. Step 3 is rejected with HTTP 403 Forbidden ("Moderators may only edit users in their own department").
- **Actual result:** Step 2 succeeded; Step 3 returned HTTP 403 Forbidden.
- **Status:** PASSED
- **Evidence:** `accounts/views.py` line 225 checks `target_user.department_id != request.user.department_id` and returns `HttpResponseForbidden`.

TC-12: Administrator Self-Deletion Safety Validation

- **Level / category:** Boundary / Security (*GENUINE FAILURE*)
 - **Test basis / objective:** R06 (**COND-14**) — Verify that the system prevents an Administrator from deleting their own currently logged-in account, preventing system lockout.
 - **Preconditions:** User `superadmin` is logged in as the sole active Administrator.
 - **Test data:** `superadmin` targets their own user ID on `/users/<id>/delete/`.
 - **Steps:**
 1. Log in as `superadmin`.
 2. Navigate to `/users/`.
 3. Locate `superadmin` in the user table and click "Delete".
 4. On the confirmation screen (`/users/<id>/delete/`), click "Confirm Deletion".
 - **Expected result:** The system rejects the action with a safety validation error ("You cannot delete your own active administrator account") and aborts deletion.
 - **Actual result:** The system deletes the `superadmin` account from the database and redirects to the user list. The user's active session is invalidated on subsequent requests, leaving the system without an active administrator.
 - **Status:** **FAILED**
 - **Evidence:** `accounts/views.py user_delete()` executes `target_user.delete()` unconditionally without verifying `request.user != target_user`. Logged as **BUG-02**.
-

TC-13: Department Group Deletion Blocked by Live Active Membership

- **Level / category:** System / Manual
 - **Test basis / objective:** R07 (**COND-15**) — Verify that deleting a department group is refused if active members are currently assigned to it.
 - **Preconditions:** DepartmentGroup "Physics Department" exists and contains 2 assigned users.
 - **Test data:** Group ID for "Physics Department".
 - **Steps:**
 1. Log in as Administrator.
 2. Navigate to `http://127.0.0.1:8000/groups/`.
 3. Click "Delete" next to "Physics Department".
 4. On the confirmation screen, click "Confirm Delete".
 - **Expected result:** Deletion refused; error banner displayed: "Cannot delete group: group still has members. Reassign or remove members first." Group remains intact in database.
 - **Actual result:** Live check `group.members.exists()` evaluated to `True`; deletion aborted; error displayed on screen.
 - **Status:** **PASSED**
 - **Evidence:** Database query confirms "Physics Department" still exists; HTTP POST returns error message.
-

TC-14: Bulk Download ZIP Archive Generation and Extraction

- **Level / category:** System / Integration
- **Test basis / objective:** R04 (**COND-10**) — Verify that bundling multiple search results produces a valid, uncorrupted ZIP archive containing publication files.
- **Preconditions:** Multiple publications with uploaded PDF attachments exist in the database.
- **Test data:** Query selecting 3 distinct publications.
- **Steps:**
 1. Execute search returning 3 publications.
 2. Click "Download All" (`/search/download-all/`).
 3. Inspect HTTP response headers and stream payload.
 4. Decompress the downloaded archive using Python `zipfile`.
- **Expected result:** Response has MIME type `application/zip` and filename `publications_archive.zip`; archive decompresses cleanly into individual files matching publication titles.

- **Actual result:** Archive received, MIME type verified, and decompressed successfully without CRC or header corruption.
- **Status:** **PASSED**
- **Evidence:** **FileResponse** streamed 200 OK; extracted zip contains expected files plus text summaries for abstract-only entries.

Table C — Traceability Record

Requirement	Condition	Test Case	Execution Result	Defect Report
R01 (FR)	COND-01	TC-01 (Manual System)	PASSED	N/A
R01 (FR)	COND-02	TC-02 (Invalid Input)	PASSED	N/A
R01 (FR)	COND-03	TC-01 (Manual System)	PASSED	N/A
R01 (FR)	COND-04	TC-04 (Exact Boundary)	PASSED	N/A
R02 (FR)	COND-05	TC-03 (Security Rules)	PASSED	N/A
R02 (FR)	COND-06	TC-05 (System / Network)	PASSED	N/A
R03 (FR)	COND-07	TC-07 (Exact Boundary)	PASSED	N/A
R03 (FR)	COND-08	TC-06 (Boolean Syntax)	FAILED	BUG-01
R04 (FR)	COND-09	TC-05 (File Download)	PASSED	N/A
R04 (FR)	COND-10	TC-14 (Bulk Bundling)	PASSED	N/A
R05 (FR)	COND-11	TC-08 (Manual System)	PASSED	N/A
R05 (FR)	COND-11	TC-09 (Invalid Input)	PASSED	N/A
R05 (FR)	COND-12	TC-10 (Publisher Edit)	BLOCKED	N/A (<i>Precondition Blocked</i>)
R06 (FR)	COND-13	TC-11 (Dept Isolation)	PASSED	N/A
R06 (FR)	COND-14	TC-12 (Admin Lifecycle)	FAILED	BUG-02
R07 (FR)	COND-15	TC-13 (Manual System)	PASSED	N/A

Part 4 — Defect Reporting and Final Quality Judgment (15 Marks)

Defect Investigation Process

Every non-passing test case was investigated before determining whether it represented an authentic defect:

- **TC-10 (BLOCKED):** Investigated as a potential defect. Analysis revealed that the inability to edit was caused by an upstream administrative account lock. Because the authentication gate correctly enforced security policy by refusing locked credentials, this represents an expected operational dependency rather than a software defect. It is properly classified as **BLOCKED** and not logged as a Jira defect.
- **TC-06 (FAILED):** Investigated against the test basis (SRS §2.3.2). Confirmed that the SRS explicitly mandates boolean operators (**AND**, **OR**, **NOT**). The codebase only executes literal substring matching. This is a reproducible implementation divergence and is logged as **BUG-01**.
- **TC-12 (FAILED):** Investigated against administrative security best practices. Confirmed that deleting one's own active user account creates an orphaned session state and risks total administrative lockout. This is a confirmed implementation defect and is logged as **BUG-02**.

Jira Defect Records

Jira Defect #1: BUG-01

- **Issue Key / ID:** PMS-BUG-01
- **Issue Type:** Bug / Defect
- **Summary / Title:** Publication search fails to parse boolean logical operators (AND, OR, NOT)
- **Affected Environment / Build:** Local Development / Build v1.0.0-frozen (Python 3.14, Django 5.2.17, Windows 11)
- **Preconditions:** Multiple publications exist in the database with overlapping keywords (e.g., "Software Testing in Practice").
- **Minimal Reproduction Steps:**
 1. Navigate to `/search/` while authenticated or in VUB Network Mode.
 2. In the "Keywords" input field, type: `Software AND Testing`.
 3. Click "Search Publications".
- **Expected Result:** Per SRS §2.3.2, the search engine parses the AND operator, matches records containing both tokens, and displays matching results.
- **Actual Result:** The page displays "No publications match your search criteria" because the ORM executes a literal substring match for `"Software AND Testing"`.
- **Reproducibility:** 100% Reproducible.
- **Severity:** Major (breaks core functional discovery requirement specified in SRS).
- **Priority:** High.
- **Related Test Case ID:** TC-06
- **Supporting Evidence:** Code inspection of `publications/services.py` lines 24–27:

```
keywords = (criteria.get("keywords") or "").strip()
if keywords:
    qs = qs.filter(Q(title__icontains=keywords) | Q(abstract__icontains=keywords))
```

- **Workflow Status:** Open / Ready for Review.

Jira Defect #2: BUG-02

- **Issue Key / ID:** PMS-BUG-02
- **Issue Type:** Bug / Security Defect
- **Summary / Title:** Administrator can delete own active user account causing session invalidation and potential lockout
- **Affected Environment / Build:** Local Development / Build v1.0.0-frozen (Python 3.14, Django 5.2.17, Windows 11)
- **Preconditions:** Administrator is logged in with an active session (`Role.ADMINISTRATOR`).
- **Minimal Reproduction Steps:**
 1. Log in as an Administrator.
 2. Navigate to the User Administration list at `/users/`.
 3. Find the currently logged-in administrator's account and click "Delete".
 4. On the confirmation screen (`/users/<id>/delete/`), click "Confirm Deletion".
- **Expected Result:** The application prevents self-deletion with an error banner: "You cannot delete your own administrator account."
- **Actual Result:** The user record is deleted immediately from the database; the active administrator session is orphaned; the system can be left with zero administrative accounts.
- **Reproducibility:** 100% Reproducible.
- **Severity:** Critical (administrative integrity and denial-of-service vulnerability).
- **Priority:** Highest.
- **Related Test Case ID:** TC-12
- **Supporting Evidence:** Code inspection of `accounts/views.py` lines 273–281 shows unconditional deletion without self-identity checks:

```
@require_role("administrator")
def user_delete(request, pk):
    target_user = get_object_or_404(User, pk=pk)
    if request.method == "POST":
        target_user.delete() # Missing: if target_user == request.user: abort
```

- **Workflow Status:** Open / Ready for Review.

Final Quality Judgment (300–400 words)

The comprehensive quality evaluation of the Publications Management System (PMS v0.10) indicates that the frozen baseline implementation satisfies fundamental architectural and security objectives, but requires targeted remediation prior to production deployment. Empirical test execution across the 10 selected requirements demonstrated robust enforcement of core access control policies, entity lifecycles, and cryptographic standards. Specifically, the six-level user hierarchy, departmental isolation for moderators, PBKDF2 password hashing, and live relational constraints blocking the deletion of non-empty department groups performed reliably with zero regressions across 45 automated and manual test cases.

However, critical functional and security deficiencies remain unsupported. Functional testing revealed that the search subsystem (R03) implements raw substring matching rather than the boolean query grammar (**AND**, **OR**, **NOT**) explicitly mandated by SRS §2.3.2, causing valid multi-token literature searches to fail. Furthermore, the administrative user management interface (R06) lacks basic identity self-checks, permitting administrators to delete their own active credentials and risk institutional lockout. Static analysis via SonarQube reinforced these concerns, identifying critical security hotspots including hardcoded development secret keys and broad exception-swallowing patterns that mask low-level I/O failures during bibliographic ingestion and archive streaming.

The AI-assisted development methodology contributed significantly to these discrepancies. While generative tooling accelerated the synthesis of standard Model-View-Template boilerplate and role decorators, it introduced material, unsupported assumptions. The AI favored superficial implementations—such as ORM substring filters over recursive descent query parsers—and failed to anticipate operational edge cases, including administrator self-deletion and production secret management.

In conclusion, the evaluated 10-requirement scope is acceptable for local demonstration and functional prototyping, but cannot be considered production-ready. The codebase achieves approximately 75% defensible compliance: authentication, departmental governance, and metadata extraction are structurally sound, but release acceptance is contingent upon resolving the boolean search defect (BUG-01), enforcing administrative self-deletion guards (BUG-02), and replacing hardcoded configuration secrets with secure environment injection. *(Word count: 326 words)*

Submission Checklist & Viva Preparation Guide

Submission Checklist Confirmation

- ☒ **Pair Details & Confirmed SRS:** Documented above with PMS v0.10 selection.
- ☒ **Comprehensive Report:** Complete 10-requirement table, AI assumptions, SonarQube interpretations, testing records, traceability, and 300–400 word final judgment.
- ☒ **Frozen Baseline Source Code:** Preserved in **PMS-main** with run instructions.
- ☒ **SonarQube Evidence:** Configuration file **sonar-project.properties** provided and 5 substantive findings interpreted.
- ☒ **All 3 NFRs Evaluated:** Security, Usability, and Reliability evaluated with relevant evidence and limitations.
- ☒ **14 Executed Test Cases:** 2 exact boundary cases, 2 invalid input cases, 3 manual system cases, and 2 non-trivial cases genuinely FAILED (**TC-06**) and BLOCKED (**TC-10**).
- ☒ **Jira Defect Records:** Complete defect write-ups provided for **BUG-01** and **BUG-02**.

Oral Defense / Viva Anticipated Questions & Answers

1. Q: Why did you classify the boolean search assumption in R03 as "Unsupported"?

- **Defensible Answer:** SRS §2.3.2 explicitly specifies that search must support boolean operators (**AND, OR, NOT**). The AI-assisted code only used Django's **icontains** substring filter. Rather than hiding this omission, we classified it as unsupported, wrote a test targeting boolean syntax (**TC-06**), confirmed its genuine failure, and logged it as a defect in Jira (**BUG-01**).

2. Q: Why is TC-10 marked BLOCKED instead of FAILED, and why isn't it in Jira?

- **Defensible Answer:** A test is FAILED when the code executes and actual $\neq$ expected. A test is BLOCKED when an upstream precondition prevents execution. In **TC-10**, the account was locked by an administrator; the authentication system correctly prevented access. Since the login blocker was intended security behavior, the publication edit step could never be reached. A blocker is not necessarily a bug.

3. Q: How did you ensure your CRUD selection complied with the assignment rules?

- **Defensible Answer:** We selected exactly 3 CRUD requirements (R05 Manage Own Publications, R06 Manage User Accounts, R07 Manage Department Groups), each managing the full lifecycle of an entity. The remaining 4 FRs (R01, R02, R03, R04) represent non-CRUD behaviors such as token generation/expiration, 6-tier role authentication with network mode, search, and zip streaming.

4. Q: How does SonarQube's assessment relate to your NFR evaluation?

- **Defensible Answer:** SonarQube directly informed N01 (Security) by detecting hardcoded secrets and open debug flags, and N03 (Reliability) by flagging silent exception swallowing. However, for N02 (Usability), SonarQube is not relevant because static analysis cannot measure user click-depth or navigation flow; we used structured manual inspection instead and documented this limitation in the report.